CryptAlyst

Message-Bus Topics — Publisher & Subscriber Reference

CryptAlyst is a paper-trading crypto platform: a Go market-data feed, a RabbitMQ message bus, and a fleet of Python engines that compute signals, weigh them, and trade paper books. This document is the reference for the bus — the one broker, its eight exchanges and twenty-two topics, who publishes and who subscribes, how a model's numbers become a weighted decision, and the places where the wiring is still wrong.

How to read it: the first section defines the engine/model/book layers and the decision weighting; then the architecture counts; then the registry; then each model; then the message flow and the per-subscriber breakdown; finally the known gaps.

Contents

Contents 2
Engines, models and books — and how a number becomes a decision 3
Bus architecture — how many of each 3
Registry 3
Per-model descriptions — what each one does 4
Producer contract — what each model sends and where it is inserted 6
Message flow — how a model's output travels 8
Pseudo-code 8
Subscriber breakdown 8
Models not receiving the topics they should listen for 11

Engines, models and books — and how a number becomes a decision

Engine = a running process (the supervisor launches one `python3 -u <file>.py`). **Model** = a strategy/rule inside an engine — one engine can run many models, and `--only` narrows it. **Book** = the paper-trading account for one model: its equity, open positions and closed trades. One book per model. A model's numbers land on its book.

When a model has finished computing, its numbers are not a trade — they are a **signal** that then passes a chain of gates and weights before the book opens a position.

1 · the model scores. The model's rule reads its inputs and returns a **side** (buy/sell) and a **score** (its conviction). **2 · the forecast.** The 7-agent forecaster blends momentum/volume/volatility/reversion/ma/breakout/pressure into one **probability (0–1)** plus an **expected move**. **3 · the TA.** tad supplies signal votes (EMA cross, RSI, Donchian). **4 · the OI gate.** the price×OI quadrant multiplies the score, strategy-aware. **5 · the gates.** forecast_min (0.45), min_expected_move, allow_counter_trend, and the higher-timeframe trend gate (ext_gate) each veto or admit. **6 · the risk gate.** risk.allow checks max_positions, confidence and signal age. **7 · the entry.** if every gate passes, the book opens a position, sized by the weighting below.

How the weighting works. The forecast is the primary weight: its blended probability must clear forecast_min to enter at all. Conviction sizing scales capital with the number of confirmations — `risk_for(base, conf) = base × (0.6 + 0.2 × min(conf, 4))`. The OI quadrant is a **strategy-aware multiplier** on the score (momentum prefers confirmation, reversion prefers divergence), and `size_mult` (now 3.0) scales the final notional. No single number is decisive — the decision is the product of the model score, the forecast probability, and these multipliers.

Bus architecture — how many of each

Layer	Count	Description
Broker	1	RabbitMQ, STOMP :61613 — the single message backbone
Exchanges	8	market, intel, alert, model, risk, review, exec, system (routing groups)
Topics	22	routing keys within the exchanges (the full registry above)
Queues	~60	2 per topic (q. for bus_consumer, q.rec. for recorder) + stomp-subscription-* for live subscribers
Producers	14	market_data, tad, forecast, signals, feeds, market_agent, portfolio_review, books, backend, supervisor, healthpub, master, cli, maker
Consumers	5	recorder (:8801), bus_consumer (:8798), backend (:8787), signal_bus (:8793), master (:8799)
Secondary bus	1	signal_bus :8793 is an HTTP proxy (/latest) the books poll — not a second RabbitMQ broker

Registry

Topic	Exch	Ret	Publisher(s)	Other subscriber(s)
`market.price`	market	drop	market_data	backend (turtle), master
`market.candle`	market	drop	market_data	tad (TA), forecast (warm-up)
`market.depth`	market	drop	market_data	backend (detectors), feeds
`market.mini24h`	market	drop	market_data	master (24h change)
`ticks`	market	drop	market_data	backend (flow windows)
`intel.signal`	intel	60s	signals	books (crypto_pack / executor / alpha via :8793)
`intel.flow`	intel	60s	signals	books (flow confidence)
`intel.movers`	intel	60s	market_data, signals	books (movers surface)
`intel.board`	intel	60s	market_agent	books, master (opportunity board)
`intel.ta`	intel	60s	tad	books (ta_vote via ta_live.json)

Topic	Exch	Ret	Publisher(s)	Other subscriber(s)
intel.forecast	intel	60s	forecast	books (forecast gate)
alert.pumpdump	alert	300s	backend, signal_bus	fast_ignition, crypto_strategies
model.decision	model	5m	alpha_rank, models10, crypto_pack, executor, alpha_signals, skim_pump, curve_rider, hft, movers, grid	master (journal)
model.fill	model	5m	same as decision	master, fit_weights (research)
model.pnl	model	5m	decision producers + supervisor	master, capital (allocator)
model.rank	model	5m	alpha_rank	master
model.evaluate	model	60s	tad	books (signal-completion pacer)
risk.state	risk	5m	risk	books (risk gate)
review.verdict	review	5m	portfolio_review	master
exec.quote	exec	60s	maker	
system.health	system	10m	healthpub, master	master
system.control	system	1d	master, supervisor, cli	master

Per-model descriptions — what each one does

market_data (Go :8796)

How it works — The single process allowed to talk to Binance. It subscribes to the exchange websocket (bookTicker, depth5, kline_1m/15m, miniTicker) and shards the streams — at most 900 per connection — so ~400 symbols are carried on several independent sockets. On startup it BACKFILLS 200 × 1m and 200 × 15m bars per symbol from REST, so the platform is never history-starved after a restart. It serves REST endpoints (/klines, /movers, /streamprices) and publishes the normalised topics.

Best used for — The raw truth everything else derives from. Books must never hit Binance directly — they read these topics or the REST endpoints. The backfill is what made the platform start trading again after a fresh boot.

tad (Go :8792)

How it works — The technical-analysis daemon. It pulls 1m klines per symbol and computes EMA (9/21/50), RSI14, ATR14, VWAP, a 20-bar Donchian channel and a volume-spike ratio, then emits the signals that actually FIRE — EMA crosses, overbought/oversold, breakouts, VWAP stretch, trend vs EMA50. Since the per-symbol rebuild it recomputes only when a symbol's own bar closes (no batch sweep), and it paces the model-evaluate pulse to the books.

Best used for — Its intel.ta signals are the TA vote in the decision chain — a book reads them as confirmation, and the 60s staleness turns an absent vote into a no-vote. Its model.evaluate pulse is the pacer that should wake each book when a symbol's signal set completes.

forecast (Go :8790)

How it works — Seven agents look at the same 1m series from a different angle — momentum (EMA slope + recent return), volume, volatility, reversion, MA, breakout, pressure — and each returns a probability (a sigmoid of its raw score) plus an expected move. A supervisor BLENDS them into one probability and move estimate, and a grade() feedback loop reweights each agent by whether its past calls made money. It refuses to answer below MIN_BARS=50 (warm-up).

Best used for — The primary weight in a decision: its blended probability must clear forecast_min (0.45) to enter, its move must clear min_expected_move, and consensus_dead closes a losing position only when the blend AND a majority of agents turn against the thesis.

signals (Python)

How it works — Pulls the most-traded names from marketdata, then computes each symbol's taker buy/sell ratio (long/short pressure) and a flow confidence from its recent flow series. It also republishes the movers list so that surface reaches the bus.

Best used for — Its intel.flow feed is the conviction input to sizing (risk_for scales capital with flow confidence); intel.signal is the entry signal for the signal-consuming books.

feeds (Python)

How it works — The data the models could not see before. It polls open interest, the order book and the taker ratio, keeps a 1-hour ring buffer of OI and mid-price per symbol, and computes the OI change over 5m/15m/1h plus the price×OI quadrant (confirmation/divergence/accumulation/distribution/climax).

Best used for — Its features.json feeds the OI gate — the quadrant is a strategy-aware score multiplier (momentum prefers confirmation, reversion prefers divergence), and it is the substrate for the F4 with/without-gate replay.

market_agent (Python)

How it works — Watches the liquid universe and scores every name, then ranks the top opportunities and flags the ones above a score threshold. It writes the board to disk AND publishes it to intel.board.

Best used for — The owner-critical opportunity board the owner reviews every cycle — and alpha reads its flagged names to know which symbols to trade.

alpha · funding_carry, xsec_momentum, cascade_rebound (Python)

How it works — Three fee-viable books, each its own strategy: xsec_momentum is cross-sectional — long the strongest N and short the weakest N by 24h return, rebalanced every 15m, market-neutral so BTC beta cancels and only relative alpha remains. cascade_rebound buys the snap-back after a 1m drop of 3 sigma on 5× median volume (a liquidation proxy without the feed). funding_carry fades a perpetual-funding extreme while spot is flat, collecting the reversion over ~8h. Each reads the board, klines, funding, forecast and TA, then trades through the shared gates.

Best used for — The owner-selected winners. Their model.decision/fill/pnl are the realised PnL the leaderboard and the solver measure. Each book's tunables live in its CFG (tp/sl/hold/risk).

models10 · funding_fade (Python)

How it works — Reads the funding premium (premiumIndex). If the funding rate is above +0.08% it shorts — the crowd is paying heavily to stay long, so the reversion is fadable; below -0.08% it buys. It is gated by the forecast and the higher-timeframe trend, and sized by RISK with the 8%-of-book cap scaled by size_mult.

Best used for — The kept funding book. Its signal is a mean-reversion on the funding premium; the forecast and trend gates stop it fading into a regime that is still trending.

crypto_strategies · model_stack (Python)

How it works — Sums the model votes for a symbol; when at least two agree ($|net| \geq 2$) it trades the consensus — long if the net is positive, short if negative. It reads the alert and model-vote streams from the Go backend.

Best used for — A consensus filter: it only acts when several independent models point the same way, which is meant to cut the single-model noise that Stage-0 found costs the book.

churn (Python)

How it works — A throughput demo, honestly labelled: one fill per second regardless of direction, alternating side. It exists to prove the whole pipeline (signal → decision → fill → ledger) moves, not as an edge — at 0.2% round-trip its fees alone bleed the book.

Best used for — A canary for the pipeline, not a strategy. Its fills exercise the decision→fill→ledger chain so the substrate stays honest.

backend · turtle / detectors (Go :8787)

How it works — The Go detector server: it consumes market flow/depth/ticks and runs the pump/dump onset and turtle-breakout detectors, emitting alerts plus the screen/survey surfaces.

Best used for — Its alert.pumpdump feed is the alert stream fast_ignition and crypto_strategies act on; the screen/survey surfaces are the owner's watch view.

recorder (Go :8801)

How it works — The single bus consumer and the only writer of the lake. Every envelope passes the interceptor pipeline — authority, keying, correlation, freshness — each recorded as a hop, then is written once to a rotating, indexed, compressed segment.

Best used for — The durable record (state/lake3) every research artefact reads: the F4 replay, the solver, fit_weights and the decision journal all replay this lake. Nothing is ever deleted — segments seal, never truncate.

portfolio_review (Python)

How it works — Grades every book (trade count, return_pct, recent-loss streak) and stops the underperformers, UNION-adding them to the disable list so it never clobbers the owner's config. Publishes each verdict.

Best used for — The book lifecycle: it removes books that have had enough trades to judge and still lose, and its review.verdict feed is the audit trail of those stops.

Producer contract — what each model sends and where it is inserted

Wire format: Python producers pass the registry gate (topic registered AND producer in its allow-list), then wrap the payload in an envelope `{topic, key, producer, data, version:1, seq, ts}` and SEND it to `/exchange/<exchange>/<topic>` on RabbitMQ :61613 (lib/bus.py:101-126,176-198). Go producers send the raw payload body (marketdata adds `v, plane, topic, producer, key, gen, seq, cid, computed_at` headers, stomp.go:183-204); Python consumers wrap it on receipt when the body has no topic field (bus.py:307-325). The exchange fans each message out to every bound queue — the recorder's durable mirror, bus_consumer's projection, and each live subscriber's private queue.

Topic	What the model produces	Data in the envelope	Inserted at (publish call sites)
`market.price`	Last trade price plus 24h-change metadata for the whole liquid universe, one compact map per tick.	`{p: {sym: px}, m: {sym: {c: pct24h, q: quoteVol}}, ts, msgs} — Go variant: {p, ts}`	`market_data.py:292 core/cmd/marketdata/main.go:228`
`market.candle`	Closed 1m/15m OHLCV bars only (k.K.X) — the TA and forecast warm-up feed.	`{symbol, interval, t, o, h, l, c, v, q, Q, ts}`	`market_data.py:249 core/cmd/marketdata/main.go:589→426`
`market.depth`	5-level order book per symbol with sizes, spread and bid/ask imbalance; published on change or 1s heartbeat.	`py: {symbol, bids: [[p,q]×5], asks: [[p,q]×5], spread_bps, bid_qty, ask_qty, imbalance, ts} — Go: bids/asks as [{price,qty}]`	`market_data.py:328-333 core/cmd/marketdata/main.go:243`
`market.mini24h`	Universe-size heartbeat — how many symbols the 24h mini-ticker window covers.	`{count, ts}`	`market_data.py:246 core/cmd/marketdata/main.go:229-230`
`ticks`	All-symbol tick snapshot (price + 24h move + quote vol); the flow-window input on the backend.	`{p: {sym: px}, m: {sym: {pct, qv, p}}, ts} — backup path: {p, ts, msgs} raw, no envelope`	`market_data.py:293 core/cmd/marketdata/main.go:227 market_data_rest_backup.py:304 (raw, bypasses registry)`
`intel.signal`	signals.py per-symbol analysis score — taker-buy ratio, flow confidence and parts blended into one rankable number.	`{symbol, taker_by_ratio, flow_conf, flow_parts, score, ts} (from analyse())`	`signals.py:325`
`intel.flow`	Flow confidence per symbol with the last 10 flow readings — the books' flow-conviction input.	`{symbol, buy_ratio, flow_conf, flow_series: last 10, score}`	`signals.py:326-332`
`intel.movers`	Gainers/losers/most-traded surface. Two producer variants with different payload shapes.	`signals: {gainers, losers, most_traded, universe, ts} — market_data (slimmer): {gainers: [{s,pct,qv}]×15, losers, ts}`	`signals.py:306 market_data.py:269-271`

Topic	What the model produces	Data in the envelope	Inserted at (publish call sites)
intel.board	The market_agent opportunity board: top-15 ranked rows with score components, its natural-language thoughts, and flagged symbols. Owner-critical.	{ts, universe, opportunities: rows[:15], thoughts: [{kind,sym,score,text,ts}][:20], flagged: [sym]}	market_agent.py:127
intel.ta	tad per-symbol TA snapshot — the full signal set per cycle, or a single firing signal.	{symbol, signals: [sig], count, ts} or {signal: sig, ts}; sig = {cursor, symbol, kind, side, strength, note, ts}	tad/main.go:317 tad/main.go:326 tad/main.go:414 (/scan)
intel.forecast	forecast service blended forecast per symbol+side+horizon with agent views and a trade action (enter/hold/liquidate). Gates book entries.	{symbol, side, horizon_s, prob, move_pct, confidence, action, resolution, views: [{agent, side, prob, move_pct, confidence, note}], price, bar_age_s, ts}	forecast/main.go:502 forecast/main.go:565
alert.pumpdump	Pump/dump alert from the backend detectors with severity, reasons and full metrics; signal_bus republishes with cursor+received_ms.	{symbol, kind, score, event_id, severity, summary, ts_ms, detector, reasons: []} + {cursor, received_ms} on republish	signal_bus.py:33-50 core/cmd/signalbus/main.go:75
model.decision	A book's entry decision, chained by decision_id to every later fill. Emitted through the single lib/emit emitter by every book that opens a position.	{decision_id, engine, symbol, side, allowed, reason, confidence, ts_ms}	lib/emit.py:85 ← models10.py:268, crypto_strategies.py:222, movers.py:263, hybrids.py:287, alpha_rank.py:172, grid.py:218, churn.py:76, hft.py:280, alpha_signals.py:363
model.fill	A closed position's result, carrying fill_id + the originating decision_id — the decision-outcome substrate.	{fill_id, decision_id, engine, symbol, side, entry, exit, gross_pct, net_pct, peak_pct, reason, held_s, notional, pnl_usd, entry_ms, exit_ms, features, rank_score, why, slippage_bps, signal_price}	lib/emit.py:61 ← models10.py:299, crypto_strategies.py:173,213, movers.py:230, hybrids.py:311, alpha_rank.py:125, grid.py:253, churn.py:66, hft.py:249, alpha_signals.py:180,223,272
model.pnl	Equity mark: alpha_rank on each position close; the supervisor on each cycle for every graded book.	alpha_rank: {engine, equity, net_pct, symbol} — supervisor: {engine, equity, return_pct, trades, open}	alpha_rank.py:111-113 supervisor.py:317-320
model.rank	alpha_rank cross-sectional ranking — top and bottom of the liquid universe by blended score.	{top: [{symbol, score}]×8, bottom: [...]×8, ts} (key 'xsec')	alpha_rank.py:134-137
model.evaluate	tad's bar-close pacer pulse with a fresh feature snapshot — the book-decision layer's completion signal. Currently NO subscriber.	{symbol, reason: 'bar_close', ts_ms, inputs: fresh.snapshot(sym, now)}	tad/main.go:363
risk.state	risk.py portfolio snapshot — gross exposure, direction counts, per-symbol notionals; the shared gate's view of the books.	{snapshot: {equity, start_equity, return_pct, gross_notional, gross_multiple, longs, shorts, per_symbol: {sym: notional}}, ts}	risk.py:185

Topic	What the model produces	Data in the envelope	Inserted at (publish call sites)
`review.verdict`	portfolio_review's per-book verdict each cycle — stop for evidence-losing books, keep for the rest. Its disable write is a read-modify-UNION (A6).	`{engine, action: stop|keep, why, return_pct}`	`portfolio_review.py:25-34 (pub fn) portfolio_review.py:102 (call)`
`exec.quote`	maker.py quote intent — the paper execution layer's priced quote per symbol+side.	`{symbol, side, px, prob, notional}`	`maker.py:49-50`
`system.health`	Platform health snapshot — every service's up/down, latency and port; the dashboard's health panel source.	`{services: [{name, up, ms, detail?, port}], up, total, worst_ms, ts}`	`health_pub.py:70 core/cmd/healthpub/main.go:97`
`system.control`	Control plane for operator commands (restart/stop/flag). WARNING: the only publish site is malformed (TypeError swallowed at master.py:241; Bus has no close()) — the topic is effectively unsent.	`{action, actor: 'dashboard', ts} — intended shape`	`master.py:241 (BROKEN — never sends)`

Message flow — how a model's output travels

A model (book) computes a result and publishes it to **one topic**. The bus wraps it in an **envelope** and routes it by exchange + routing key. Every consumer receives the envelope and reads **meta.key** (the symbol) to act on the right instrument — routing is by key, never by broadcast.

Aspect	Envelope fields
The model REQUIRES (reads from an inbound envelope)	meta.key (the symbol), meta.topic, data (the payload) — that is all a consumer needs to route and act
The model ADDS (when it publishes)	producer (its name), data (its calculation), key (the symbol), plus CID / refs (the decision→fill provenance)
The model COMMUNICATES BACK	one topic it publishes to (e.g. model.decision / model.fill) carrying the payload

Pseudo-code

```
# 1 · a model computes a result
side, conf = funding_fade_strategy(klines("BTCUSDT")) # -> ("buy", 0.7)

# 2 · it publishes ONE result to ONE topic, tagged with the symbol
emit.decision("models10", "BTCUSDT", "buy", True, "funding_fade")
-> bus.pub("model.decision", "BTCUSDT", payload, producer="models10")
-> spec = TopicRegistry["model.decision"] # exchange "model"
-> guard: producer "models10" must be in spec.producers # else REFUSED
-> env = Envelope(topic="model.decision", key="BTCUSDT",
producer="models10", data=payload) # stamps version+received_at
-> transport.publish(exchange="model", routing_key="model.decision", env.json)

# 3 · the broker fans out to the queues
# q.model.decision (bus_consumer) · q.rec.model.decision (recorder)
# stomp-subscription-* (master + other live subscribers)

# 4 · the recorder runs the interceptor pipeline, then writes the lake
for rule in [authority, keying, correlation, freshness]:
env = rule(env); env.meta.hops.append(hop(rule, outcome)) # pass | refuse

# 5 · the correct engine receives it and routes by key (the symbol)
def on_model_decision(env):
sym = env.meta.key # "BTCUSDT" -> the engine acts on THAT symbol
data = env.data # the decision payload
book.consider(sym, data) # no broadcast; the key selects the symbol
```

Subscriber breakdown

recorder (:8801) and **bus_consumer (:8798)** subscribe to every topic; the breakdown lists them per topic plus each topic-specific consumer with its usage and code location.

Topic	Subscriber	Usage	Code (file:line)
market.price	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	backend (turtle)	turtle book pricing	backend/internal/turtle/store.go
market.candle	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	tad	TA bars (1m)	tad/main.go:62
	forecast	warm-up bars	forecast/main.go:89
market.depth	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	backend (detectors)	depth/pump detectors	backend/internal/detect/*.go
	feeds	book imbalance	feeds.py:74
market.mini24h	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	master	24h change surface	master.py
ticks	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	backend (flow)	flow windows	backend/internal/flow/*.go
intel.signal	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	books	entry signals via :8793 /latest	crypto_pack.py:182, executor.py:41
intel.flow	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	books	flow confidence	crypto_pack.py:182
intel.movers	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	books	movers surface	crypto_pack.py:182
intel.board	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	books / master	opportunity board	market_agent.py:126 (pub)
intel.ta	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	tactics.ta_signals	TA vote → ta_live.json	tactics.py:362

Topic	Subscriber	Usage	Code (file:line)
`intel.forecast`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	books	forecast gate	`tactics.py forecast()`
`alert.pumpdump`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	fast_ignition	alert-scored entries	`fast_ignition.py`
	crypto_strategies	pump/dump signals	`crypto_strategies.py:12`
`model.decision`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	decision journal	`master.py:1009`
`model.fill`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	fill journal	`master.py:1009`
	fit_weights	research weights	`fit_weights.py`
`model.pnl`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	pnl surface	`master.py:1009`
	capital	allocator input	`capital.py`
`model.rank`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	rank surface	`master.py:1009`
`model.evaluate`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	books	signal-completion pacer pulse	`tad/main.go:320 (pub)`
`risk.state`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	books	risk gate	`risk.py`
`review.verdict`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	review surface	`master.py:1009`
`exec.quote`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
`system.health`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	health panel	`master.py:7239`

Topic	Subscriber	Usage	Code (file:line)
system.control	recorder (:8801)	single lake writer (all topics)	core/cmd/recorder/main.go:222
	bus_consumer (:8798)	record projection (all topics)	bus_consumer.py:64
	master	control commands	master.py

Models not receiving the topics they should listen for

These models read a **HTTP/file proxy** instead of the bus topic, or the topic has **no consumer**. The bus carries the topic; the model never subscribes to it.

Model	Should listen to	Gap	Code (file:line)
all books	model.evaluate	the signal-completion pacer pulse (published by tad) has NO subscriber — the book-decision layer is unpaced	tad/main.go:320 (pub) — no consumer
alpha (funding_carry, xsec_momentum)	intel.board	reads the opportunity board via master /api/opportunities (file proxy), not the intel.board topic	alpha.py:175
crypto_strategies (model_stack)	alert.pumpdump	reads /api/alerts from backend :8787 (HTTP), not the alert.pumpdump topic	crypto_strategies.py:12
fast_ignition (disabled)	alert.pumpdump	reads the alert stream via backend HTTP, not the topic	fast_ignition.py
models10 (funding_fade)	intel.ta	reads TA via ta_live.json (tactics.ta_signals), not the intel.ta topic	tactics.py:362
models10 (funding_fade)	intel.forecast	reads forecast via FC_API :8790, not the intel.forecast topic	models10.py:66
crypto_pack / executor (disabled)	intel.signal / intel.flow / intel.movers	the signal consumers are disabled under best-5, so these topics have no live subscriber	crypto_pack.py:182, executor.py:41